Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

# Структури даних та алгоритми
# Лабораторна робота №4

«Алгоритми двійкового пошуку»

Виконав:
студент групи ІО-61
Астаф'єв Олександр Владиславович
Номер у списку групи: 2
Перевірив:
Русінов Володимир Володимирович

Київ-2026

**Мета лабораторної роботи:**

Засвоєння теоретичного матеріалу та набуття практичних навичок розв'язання задачі пошуку заданої категорії елементів за допомогою різних алгоритмів методу двійкового пошуку у двовимірних масивах.

**Постановка задачі:**

1. Написати програму розв'язання задачі пошуку (за варіантом) у двовимірному масиві (матриці) методом двійкового пошуку. Конкретний алгоритм двійкового пошуку (№1 або №2) задається варіантом завдання.
2. Розміри матриці m та n взяти самостійно в межах від 7 до 10.
3. При тестуванні програми підбирати такі вхідні набори значень матриці, щоб можна було легко відстежити коректність виконання пошуку, і щоб ця коректність була протестована для всіх можливих випадків. З метою тестування дозволяється використовувати матриці меншого розміру.

2 Задано матрицю дійсних чисел A[m,n]. Окремо у кожному стовпчику матриці визначити присутність заданого дійсного числа X і його місцезнаходження (координати) методом двійкового пошуку (**Алгоритм №2**), якщо елементи кожного стовпчика окремо впорядковані **за незбільшенням**.

**Текст програми:**

```
#include <stdio.h>


#define M 9

#define N 9


int binary_search_2(float A[M][N], int col, float x, int *steps)

{

   int L = 0, R = M - 1, i, res;


   *steps = 0;

   while (L < R)

   {

      i = (L + R) / 2;
```

```
        (*steps)++;
        if (A[i][col] > x)
            L = i + 1;
        else
            R = i;
    }
    if (A[R][col] == x)
        res = R;
    else
        res = -1;
    return res;
}

int main()
{
    float A[M][N] = {
        {35.0, 30.0, 32.5, 31.0, 34.0, 33.5, 35.0, 29.5, 32.0},
        {35.0, 26.5, 27.0, 25.5, 29.0, 28.0, 30.5, 29.5, 26.0},
        {35.0, 26.5, 21.5, 25.5, 24.0, 28.0, 23.5, 29.5, 19.5},
        {28.5, 26.5, 16.0, 20.0, 24.0, 28.0, 18.0, 29.5, 19.5},
        {22.0, 26.5, 16.0, 14.5, 24.0, 21.0, 18.0, 20.5, 19.5},
        {22.0, 19.0, 16.0,  9.0, 17.5, 21.0, 12.0, 14.0, 19.5},
        {15.5, 12.5, 16.0,  3.5, 11.0, 21.0, 12.0,  7.5, 10.0},
        { 9.0,  5.0,  8.5,  3.5,  4.5,  6.0, 12.0,  2.0,  4.0},
        { 2.5, -3.0,  1.0,  3.5, -2.0,  0.0, 12.0, -1.5,  0.0}
```

```
};
float x;
int i, j, r, steps, total = 0, found = 0;

printf("Matrix A[%d,%d]:\n", M, N);
for (i = 0; i < M; i++)
{
    for (j = 0; j < N; j++)
        printf("%6.1f", A[i][j]);
    printf("\n");
}

printf("\nEnter X: ");
scanf("%f", &x);
printf("\n");

for (j = 0; j < N; j++)
{
    r = binary_search_2(A, j, x, &steps);
    total += steps;
    if (r != -1)
    {
        printf("A[%d][%d] = %.1f;\n", r + 1, j + 1, x);
        found = 1;
    }
```

```
    }
    if (!found)
        printf("Number %.2f not found.\n", x);

    printf("\nBinary search steps: %d (linear search: up to %d)\n", total, M * N);

    return 0;
}
```

**Результати програми:**

```
PS D:\Astafiev\АСД\sda-astafiev\Lab4> .\task.exe
Matrix A[9,9]:
  35.0  30.0  32.5  31.0  34.0  33.5  35.0  29.5  32.0
  35.0  26.5  27.0  25.5  29.0  28.0  30.5  29.5  26.0
  35.0  26.5  21.5  25.5  24.0  28.0  23.5  29.5  19.5
  28.5  26.5  16.0  20.0  24.0  28.0  18.0  29.5  19.5
  22.0  26.5  16.0  14.5  24.0  21.0  18.0  20.5  19.5
  22.0  19.0  16.0   9.0  17.5  21.0  12.0  14.0  19.5
  15.5  12.5  16.0   3.5  11.0  21.0  12.0   7.5  10.0
   9.0   5.0   8.5   3.5   4.5   6.0  12.0   2.0   4.0
   2.5  -3.0   1.0   3.5  -2.0   0.0  12.0  -1.5   0.0

Enter X: 32.0

A[1][9] = 32.0;

Binary search steps: 35 (linear search: up to 81)
```

```
PS D:\Astafiev\АСД\sda-astafiev\Lab4> .\task.exe
Matrix A[9,9]:
  35.0  30.0  32.5  31.0  34.0  33.5  35.0  29.5  32.0
  35.0  26.5  27.0  25.5  29.0  28.0  30.5  29.5  26.0
  35.0  26.5  21.5  25.5  24.0  28.0  23.5  29.5  19.5
  28.5  26.5  16.0  20.0  24.0  28.0  18.0  29.5  19.5
  22.0  26.5  16.0  14.5  24.0  21.0  18.0  20.5  19.5
  22.0  19.0  16.0   9.0  17.5  21.0  12.0  14.0  19.5
  15.5  12.5  16.0   3.5  11.0  21.0  12.0   7.5  10.0
   9.0   5.0   8.5   3.5   4.5   6.0  12.0   2.0   4.0
   2.5  -3.0   1.0   3.5  -2.0   0.0  12.0  -1.5   0.0

Enter X: 9.0

A[8][1] = 9.0;
A[6][4] = 9.0;

Binary search steps: 27 (linear search: up to 81)
```

```
PS D:\Astafiev\АСД\sda-astafiev\Lab4> .\task.exe
Matrix A[9,9]:
  35.0  30.0  32.5  31.0  34.0  33.5  35.0  29.5  32.0
  35.0  26.5  27.0  25.5  29.0  28.0  30.5  29.5  26.0
  35.0  26.5  21.5  25.5  24.0  28.0  23.5  29.5  19.5
  28.5  26.5  16.0  20.0  24.0  28.0  18.0  29.5  19.5
  22.0  26.5  16.0  14.5  24.0  21.0  18.0  20.5  19.5
  22.0  19.0  16.0   9.0  17.5  21.0  12.0  14.0  19.5
  15.5  12.5  16.0   3.5  11.0  21.0  12.0   7.5  10.0
   9.0   5.0   8.5   3.5   4.5   6.0  12.0   2.0   4.0
   2.5  -3.0   1.0   3.5  -2.0   0.0  12.0  -1.5   0.0

Enter X: 7.0

Number 7.00 not found.

Binary search steps: 27 (linear search: up to 81)
```

**Висновки за результатами лабораторної роботи:**

Під час виконання лабораторної роботи я засвоїв теоретичний матеріал та набув практичних навичок розв’язання задачі пошуку заданої категорії елементів за допомогою різних алгоритмів методу двійкового пошуку у двовимірних масивах. Реалізовано алгоритм двійкового пошуку №2 (найлівіший збіг, варіант 2) для матриці 9×9, кожен стовпчик якої впорядкований за незбільшенням. Пошук виконується окремо в кожному стовпчику «на місці». Алгоритм працює коректно в усіх випадках: елемент на початку стовпчика (32.0), посередині та в нижній частині (9.0), відсутній елемент (7.0). Для X = 26.5 та 19.5 (серії з 4 однакових елементів) алгоритм №2 повертає найвищий елемент серії, як і лінійний пошук. Алгоритм №1 повертає довільний елемент серії, але може завершитись раніше. Для одного стовпчика потрібно не більше $\lceil \log_2 M \rceil$ ітерацій, для всієї матриці $O(N \cdot \log M)$ проти $O(N \cdot M)$ у лінійного пошуку. У тестах виконано 27–36 порівнянь проти 81 при лінійному пошуку, тобто у 2–3 рази менше. Додаткова пам'ять не використовується.